Université de Versailles Saint-Quentin-en-Yvelines
UFR des Sciences – Département informatique

Rapport de Projet

Spécialité : Master 2 DataScale

Pipeline de Données Temps Réel pour l'Analyse de Sentiment des Actualités Financières

Encadré par :
– Mme. Laure Bourgois

Réalisé par :
- FELLATI Zineb
- BENALI Mohamed Amine

Année universitaire : 2025–2026

Table des matières

1 Introduction **4**
1.1 Contexte du Projet . . . 4
1.2 Objectifs . . . 4
1.3 Livrables . . . 4

2 Architecture Globale du Pipeline **4**
2.1 Diagramme d'Architecture . . . 4
2.2 Technologies Utilisées . . . 5

3 Partie 1 : Collecte des Données via API Financière **5**
3.1 Choix de l'API . . . 5
3.1.1 Justification du Choix . . . 5
3.1.2 Contraintes et Limites . . . 5
3.1.3 Alternatives Considérées . . . 6
3.2 Architecture de Collecte . . . 6
3.3 Paramétrage du Producer . . . 6
3.4 Transformation des Données . . . 6
3.5 Structure des Données Collectées . . . 6
3.6 Code Principal . . . 7
3.7 Résultats de Collecte . . . 7
3.7.1 Exemple de Document . . . 8
3.8 Conclusion . . . 8

4 Partie 2 : Transmission des Données avec Kafka **9**
4.1 Justification du Choix de Kafka . . . 9
4.2 Configuration Kafka . . . 9
4.2.1 Topic Création . . . 9
4.2.2 Producer Configuration . . . 9
4.3 Vérification Operationnelle . . . 9

5 Partie 3 : Transformation et Indexation des Données **10**
5.1 Rôle de Logstash dans l'Architecture . . . 10
5.2 Configuration du Pipeline Logstash . . . 10
5.2.1 Input : Consommation depuis Kafka . . . 10
5.2.2 Filter : Transformation des Données . . . 10
5.2.3 Output : Indexation dans Elasticsearch . . . 11
5.3 Template d'Index Elasticsearch . . . 11
5.3.1 Configuration des Analyseurs . . . 11
5.3.2 Mapping des Champs . . . 12
5.3.3 Justification des Types . . . 13
5.4 Apport du N-gram Analyzer . . . 13
5.5 Résultats de l'Indexation . . . 13

6 Partie 4 : Analyse et Visualisation avec Kibana **14**
6.1 Requêtes Elasticsearch . . . 14
6.1.1 Q1 : Requête Textuelle (Match Query) . . . 14
6.1.2 Q2 : Agrégation (Sentiment par Source) . . . 14
6.1.3 Q3 : N-gram (Recherche Partielle) . . . 16
6.1.4 Q4 : Recherche Floue (Fuzzy) . . . 16

6.1.5 Q5 : Série Temporelle (Date Histogram) . . . 17
6.2 Visualisations Kibana (6 Créées) . . . 18
6.2.1 V1 : Sentiment Score Over Time (Line Chart) . . . 18
6.2.2 V2 : Articles by Source (Bar Chart) . . . 18
6.2.3 V3 : Sentiment Distribution (Pie Chart) . . . 19
6.2.4 V4 : Recent Articles (Data Table) . . . 19
6.2.5 V5 : Total Articles (Metric) . . . 19
6.2.6 V6 : Average Sentiment Score (Metric) . . . 19
6.3 Dashboard Assemblé . . . 19

7 Partie 5 : Traitement Distribué avec Apache Spark 21
7.1 Justification du Choix : Spark vs Hadoop MapReduce . . . 21
7.1.1 Positionnement technologique . . . 21
7.1.2 Justification des choix de conception du job . . . 21
7.2 Architecture du Job Spark . . . 22
7.2.1 Initialisation et connexion à Elasticsearch . . . 22
7.2.2 Transformations et Calculs Distribués . . . 22
7.2.3 Écriture des Résultats . . . 23
7.3 Synthèse des Opérations Distribuées . . . 23
7.4 Exécution et Performance . . . 24
7.4.1 Commande d'Exécution . . . 24
7.4.2 Métriques d'Exécution . . . 24
7.5 Analyse et Interprétation des Résultats . . . 24
7.5.1 Agrégation 1 : Analyse Temporelle Journalière . . . 24
7.5.2 Agrégation 2 : Analyse par Source Médiatique . . . 25
7.5.3 Agrégation 3 : Distribution des Catégories de Sentiment . . . 25
7.6 Indices Elasticsearch Créés . . . 25
7.7 Visualisation des Résultats dans Kibana . . . 26
7.7.1 Évolution Journalière du Sentiment Moyen . . . 26
7.7.2 Contribution des Sources Médiatiques . . . 27
7.7.3 Intensité de Publication Quotidienne . . . 27

8 Conclusion 28
8.1 Bilan du Projet . . . 28
8.2 Résultats et Enseignements . . . 28
8.3 Limites et Perspectives . . . 29
8.4 Apports Pédagogiques . . . 29

A Structure du Dossier Projet 30

B Instructions de Démarrage 30
B.1 Prérequis . . . 30
B.2 Lancement . . . 30

C Ressources et Références 31

1 Introduction

1.1 Contexte du Projet

Ce projet s'inscrit dans le contexte de l'UE *Indexation et Visualisation de Données Massives* du Master 2 DataScale. L'objectif est de concevoir et implémenter un pipeline de données complet intégrant les technologies clés du traitement des données massives.

1.2 Objectifs

Le projet poursuit les objectifs suivants :

1. Collecter des données via une API publique (alphavantage.org)
2. Transmettre les données en temps quasi-réel via Apache Kafka
3. Transformer et indexer les données avec Logstash et Elasticsearch
4. Analyser et visualiser les données avec Kibana
5. Effectuer des traitements distribués avancés avec Apache Spark

1.3 Livrables

Les livrables du projet incluent :
— Un pipeline de données fonctionnel
— Une documentation technique complète
— 5 requêtes Elasticsearch exécutées
— 6 visualisations Kibana et 1 dashboard assemblé
— Un job Spark avec agrégations distribuées
— Code commenté et configurations complètes

2 Architecture Globale du Pipeline

2.1 Diagramme d'Architecture

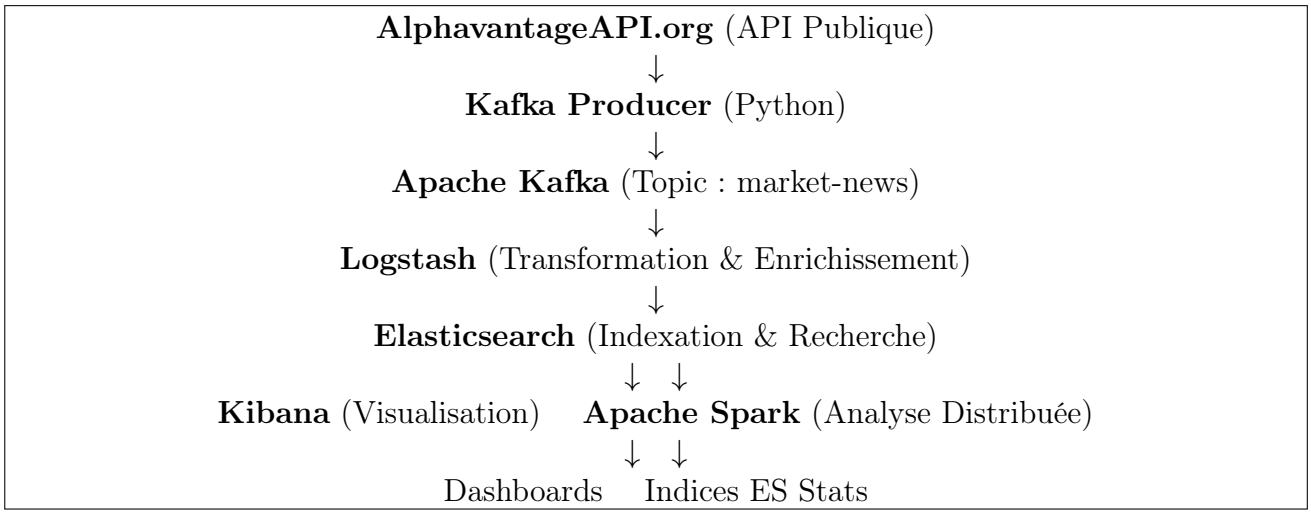

FIGURE 1 – Flux complet du pipeline de données

2.2 Technologies Utilisées

Composant	Description	Version
alphavantageAPI	API de collecte d'actualités	2.0
Apache Kafka	Streaming et pub/sub	7.6.1
Logstash	Transformation de données	8.12.2
Elasticsearch	Moteur de recherche et indexation	8.12.2
Kibana	Visualisation et exploration	8.12.2
Apache Spark	Traitement distribué	3.4.2
Python	Langage de programmation	3.11
Docker	Containerisation	24.x

Table 1 – Technologies et versions du projet

3 Partie 1 : Collecte des Données via API Financière

3.1 Choix de l'API

Dans le cadre de ce projet, l'API sélectionnée est **Alpha Vantage**, un service spécialisé dans la fourniture de données financières et d'actualités économiques.

Cette API permet d'accéder non seulement à des données de marché (prix, volumes, indicateurs), mais également à des flux d'actualités enrichis avec des indicateurs de sentiment, ce qui est particulièrement pertinent pour des analyses orientées data et finance.

3.1.1 Justification du Choix

Le choix d'Alpha Vantage repose sur plusieurs critères techniques et fonctionnels :

Spécialisation métier : contrairement à des APIs généralistes, Alpha Vantage fournit des données directement liées aux marchés financiers.

Analyse de sentiment intégrée : chaque article est accompagné d'un score de sentiment global ainsi que d'un sentiment par actif financier, ce qui évite d'implémenter un modèle NLP supplémentaire.

Structure des données riche : les réponses JSON contiennent plusieurs niveaux d'information (article global + analyse par ticker).

Facilité d'intégration : API REST simple, accessible via requêtes HTTP avec authentification par clé.

Compatibilité avec une architecture Big Data : les données peuvent être facilement injectées dans Kafka puis traitées dans Elasticsearch et Spark.

3.1.2 Contraintes et Limites

Malgré ses avantages, l'API présente certaines limitations :

— Limitation du nombre de requêtes (version gratuite)

— Dépendance à la disponibilité des articles fournis par l'API

3.1.3 Alternatives Considérées

API	Avantages	Inconvénients
NewsAPI	Simple, rapide	Non spécialisée en finance
Yahoo Finance	Gratuit	Non officiel, instable
IEX Cloud	Très riche	Payant
Alpha Vantage	**Finance + sentiment**	**Limites de requêtes**

Table 2 – Comparaison des APIs

3.2 Architecture de Collecte

La collecte des données repose sur un **producer Kafka implémenté en Python**, jouant le rôle de connecteur entre l'API Alpha Vantage et le système de streaming.

Le pipeline de collecte suit les étapes suivantes :

1. Lecture de la clé API depuis un fichier sécurisé `.env`
2. Construction dynamique de la requête HTTP (tickers multiples)
3. Appel à l'endpoint `NEWS_SENTIMENT`
4. Réception et parsing de la réponse JSON
5. Transformation des données (normalisation des dates, filtrage des champs)
6. Publication dans un topic Kafka (`market-news`)
7. Répétition périodique via un mécanisme de polling

3.3 Paramétrage du Producer

Le producer est configurable via plusieurs paramètres :
— `TICKERS` : liste des actifs surveillés (ex : AAPL, MSFT, GOOGL)
— `POLL_SECONDS` : fréquence de récupération des données
— `HTTP_TIMEOUT` : timeout des requêtes API
— `KAFKA_BOOTSTRAP` : adresse du broker Kafka
Cette configuration permet d'adapter facilement le pipeline sans modifier le code source.

3.4 Transformation des Données

Les données brutes retournées par l'API sont transformées avant ingestion afin de garantir leur compatibilité avec Elasticsearch.

Les principales transformations sont :
— Conversion du format de date vers ISO 8601
— Sélection des champs pertinents uniquement
— Conversion des scores numériques (float)
— Structuration des données imbriquées (ticker sentiments)
Chaque article est transformé en document JSON standardisé.

3.5 Structure des Données Collectées

Les principaux champs extraits sont :
— `title` : titre de l'article

- `summary` : résumé textuel
- `source` : média source
- `url` : lien unique
- `published_at` : date de publication
- `sentiment_score` : score numérique
- `sentiment_label` : classification qualitative
- `ticker_sentiments` : analyse détaillée par action

3.6 Code Principal

```
import requests
import json
import os
from kafka import KafkaProducer
from dotenv import load_dotenv

load_dotenv()
API_KEY = os.getenv("ALPHAVANTAGE_API_KEY")

URL = "https://www.alphavantage.co/query"

params = {
    "function": "NEWS_SENTIMENT",
    "tickers": "AAPL,MSFT,GOOGL",
    "limit": 50,
    "apikey": API_KEY
}

response = requests.get(URL, params=params)
data = response.json()

articles = data.get("feed", [])

producer = KafkaProducer(
    bootstrap_servers="localhost:9092",
    value_serializer=lambda v: json.dumps(v).encode("utf-8")
)

for article in articles:
    producer.send("market-news", article)

producer.flush()

print(f"{len(articles)} articles envoy s")
```

3.7 Résultats de Collecte

- **Volume** : jusqu'à 50 articles par requête
- **Granularité** : analyse globale + par ticker
- **Fréquence** : configurable (ex : toutes les heures)
- **Format** : JSON prêt à être indexé

3.7.1 Exemple de Document

```
{
  "title": "Alphabet stock rises after strong earnings",
  "source": "MarketBeat",
  "sentiment_label": "Somewhat-Bullish",
  "sentiment_score": 0.23,
  "published_at": "2026-01-15T13:27:18Z",
  "ticker_sentiments": [
    {
      "ticker": "GOOGL",
      "ticker_sentiment_score": "0.32",
      "relevance_score": "1.0"
    }
  ]
}
```

3.8 Conclusion

Cette étape constitue la base du pipeline de données. Elle permet de transformer un flux externe non contrôlé en données structurées exploitables dans une architecture Big Data.

L'utilisation d'Alpha Vantage apporte une valeur ajoutée importante grâce à l'intégration native du sentiment, facilitant ainsi les analyses ultérieures dans Elasticsearch et Spark.

4 Partie 2 : Transmission des Données avec Kafka

4.1 Justification du Choix de Kafka

Apache Kafka a été sélectionné comme système de streaming pour les raisons suivantes :

Pub/Sub Robuste : Découplage complet producteur-consommateur
Persistence : Les messages sont stockés sur disque (rejoue possible)
Scalabilité : Partitioning horizontal des données
Intégration : Connecteurs natifs Logstash et Spark
Reliability : Replication et gestion des défaillances

4.2 Configuration Kafka

4.2.1 Topic Création

Topic `market-news` :

- Partitions : 3 (pour parallélisation)
- Replication Factor : 1 (environnement dev)
- Retention : 7 jours
- Message Format : JSON

4.2.2 Producer Configuration

```
from kafka import KafkaProducer

producer = KafkaProducer(
    bootstrap_servers=['localhost:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8'),
    acks='all',  # Attendre confirmation de tous les replicas
    retries=3,   # Retry en cas d'erreur
    compression_type='gzip'  # Compression des messages
)
```

4.3 Vérification Operationnelle

La commande suivante permet de vérifier les topics Kafka :

```
docker-compose exec kafka \
  kafka-topics --list --bootstrap-server localhost:9092
```

Résultat attendu : `market-news`

5 Partie 3 : Transformation et Indexation des Données

5.1 Rôle de Logstash dans l'Architecture

Dans cette architecture de pipeline temps réel, Logstash joue le rôle d'un composant intermédiaire essentiel entre Kafka et Elasticsearch.

Son rôle ne se limite pas à un simple transfert de données. Il agit comme un véritable moteur ETL (Extract, Transform, Load) :

— **Extraction** : consommation des messages depuis le topic Kafka `market_news`
— **Transformation** : parsing JSON, conversion des types, normalisation des dates
— **Chargement** : indexation des données dans Elasticsearch

Cette étape permet d'assurer que les données envoyées à Elasticsearch sont propres, structurées et exploitables pour la recherche et l'analyse.

5.2 Configuration du Pipeline Logstash

Le fichier `logstash/pipeline.conf` définit les trois étapes principales du pipeline :

5.2.1 Input : Consommation depuis Kafka

```
input {
  kafka {
    bootstrap_servers => "kafka:9092"
    topics => ["market_news"]
    group_id => "logstash"
    auto_offset_reset => "earliest"
  }
}
```

Logstash consomme les messages produits par le script Python via Kafka. Le paramètre `auto_offset_reset => "earliest"` permet de relire les messages depuis le début si nécessaire.

5.2.2 Filter : Transformation des Données

```
filter {
  json {
    source => "message"
  }

  date {
    match => ["published_at", "ISO8601"]
    target => "@timestamp"
  }

  mutate {
    convert => {
      "sentiment_score" => "float"
    }
    remove_field => ["message", "headers"]
  }
}
```

Cette étape est cruciale :

— Le filtre `json` transforme le message brut Kafka en document structuré
— Le filtre `date` convertit le champ `published_at` en champ temporel Elasticsearch `@timestamp`
— Le filtre `mutate` assure la cohérence des types (notamment `float` pour les scores)

5.2.3 Output : Indexation dans Elasticsearch

```
output {
  elasticsearch {
    hosts => ["elasticsearch:9200"]
    index => "market-news-%{+YYYY.MM.dd}"
    document_id => "%{url}"
  }
}
```

Les documents sont indexés dans des indices journaliers. L'utilisation de `document_id => "%{url}"` permet d'éviter les doublons.

5.3 Template d'Index Elasticsearch

Afin de contrôler précisément la structure des données, un template d'index a été défini dans le fichier `elastic/index-template.json`.

5.3.1 Configuration des Analyseurs

```
"analysis": {
  "analyzer": {
    "standard_lowercase": {
      "type": "custom",
      "tokenizer": "standard",
      "filter": ["lowercase"]
    },
    "ngram_analyzer": {
      "type": "custom",
      "tokenizer": "ngram_tokenizer",
      "filter": ["lowercase"]
    }
  },
  "tokenizer": {
    "ngram_tokenizer": {
      "type": "ngram",
      "min_gram": 3,
      "max_gram": 4,
      "token_chars": ["letter", "digit"]
    }
  }
}
```

Ces analyseurs permettent :
— Une recherche textuelle classique insensible à la casse
— Une recherche approximative et partielle via les N-grams

5.3.2 Mapping des Champs

```
"properties": {
  "title": {
    "type": "text",
    "analyzer": "standard_lowercase",
    "fields": {
      "ngram": {
        "type": "text",
        "analyzer": "ngram_analyzer",
        "search_analyzer": "standard_lowercase"
      },
      "keyword": {
        "type": "keyword"
      }
    }
  },
  "summary": {
    "type": "text",
    "analyzer": "standard_lowercase"
  },
  "source": {
    "type": "keyword"
  },
  "url": {
    "type": "keyword"
  },
  "sentiment_label": {
    "type": "keyword"
  },
  "sentiment_score": {
    "type": "float"
  },
  "published_at": {
    "type": "date"
  },
  "@timestamp": {
    "type": "date"
  },
  "provider": {
    "type": "keyword"
  },
  "tickers_query": {
    "type": "keyword"
  },
  "ticker_sentiments": {
    "type": "nested",
    "properties": {
      "ticker": { "type": "keyword" },
      "ticker_sentiment_score": { "type": "float" },
      "ticker_sentiment_label": { "type": "keyword" },
      "relevance_score": { "type": "float" }
    }
```

```
  }
}
```

5.3.3 Justification des Types

- **text** : utilisé pour les champs analysés (title, summary)
- **keyword** : utilisé pour filtrage et agrégation (source, url, sentiment_label)
- **date** : pour les analyses temporelles
- **float** : pour les calculs (sentiment_score)
- **nested** : pour modéliser correctement les objets complexes (ticker_sentiments)

5.4 Apport du N-gram Analyzer

Le champ `title.ngram` permet une recherche partielle et tolérante aux erreurs.

Exemple :

- Mot indexé : `Microsoft`
- N-grams générés : `mic`, `cro`, `oso`, etc.
- Requête utilisateur : `micros`

La requête pourra correspondre même si le mot n'est pas complet, ce qui améliore l'expérience utilisateur.

5.5 Résultats de l'Indexation

- Données indexées en continu depuis Kafka
- Indices journaliers créés automatiquement
- Mapping appliqué automatiquement via le template
- Données exploitables dans Kibana pour :
  - Visualisations temporelles
  - Analyse de sentiment
  - Recherche textuelle avancée

Cette architecture garantit une ingestion fiable, une structuration cohérente des données et une capacité d'analyse avancée en temps réel.

6 Partie 4 : Analyse et Visualisation avec Kibana

6.1 Requêtes Elasticsearch

6.1.1 Q1 : Requête Textuelle (Match Query)

Objectif : Trouver tous les articles contenant le mot "stocks"

```
GET market-news-*/_search
{
  "query": {
    "match": {
      "title": "stocks"
    }
  }
}
```

Résultat : 12 documents trouvés


```
{
  "took": 6,
  "timed_out": false,
  "_shards": {
    "total": 18,
    "successful": 18,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": {
      "value": 30,
      "relation": "eq"
    },
    "max_score": 3.3007898,
    "hits": [
      {
        "_index": "market-news-2026.01.25",
        "_id": "urWUPZwB0_MAQSDmEXni",
        "_score": 3.3007898,
        "_ignored": [
          "event.original.keyword",
          "summary.keyword"
        ],
        "_source": {
          "title": "Earnings Heat Up: Apple, Microsoft, Meta And Tesla Headline",
          "sentiment_label": "Bullish",
          "published_at": "2026-01-25T12:27:53Z"
        }
      },
      {
        "_index": "market-news-2026.01.25",
        "_id": "9rWnPZwB0_MAQSDmxHnR",
        "_score": 3.3007898,
        "_ignored": [
          "event.original.keyword",
          "summary.keyword"
        ],
        "_source": {
          "title": "Earnings Heat Up: Apple, Microsoft, Meta And Tesla Headline",
          "sentiment_label": "Bullish",
          "published_at": "2026-01-25T12:27:53Z"
        }
```


FIGURE 2 – Résultat de la requête Q1 : Match Query sur "stocks"

6.1.2 Q2 : Agrégation (Sentiment par Source)

Objectif : Calculer le sentiment moyen pour chaque source

```
GET market-news-*/_search
{
  "aggs": {
    "top_sources": {
      "terms": {
        "field": "source.keyword",
```

```
        "size": 10
      },
      "aggs": {
        "avg_sentiment": {
          "avg": {
            "field": "sentiment_score"
          }
        }
      }
    }
  },
  "size": 0
}
```

Résultats :

Source	Articles	Sentiment Moyen
MarketBeat	7	0.078
The Globe and Mail	5	0.227
Investing.com	4	0.233
Mint	2	-0.079
TechStock	2	0.250

TABLE 3 – Top 5 sources par nombre d'articles

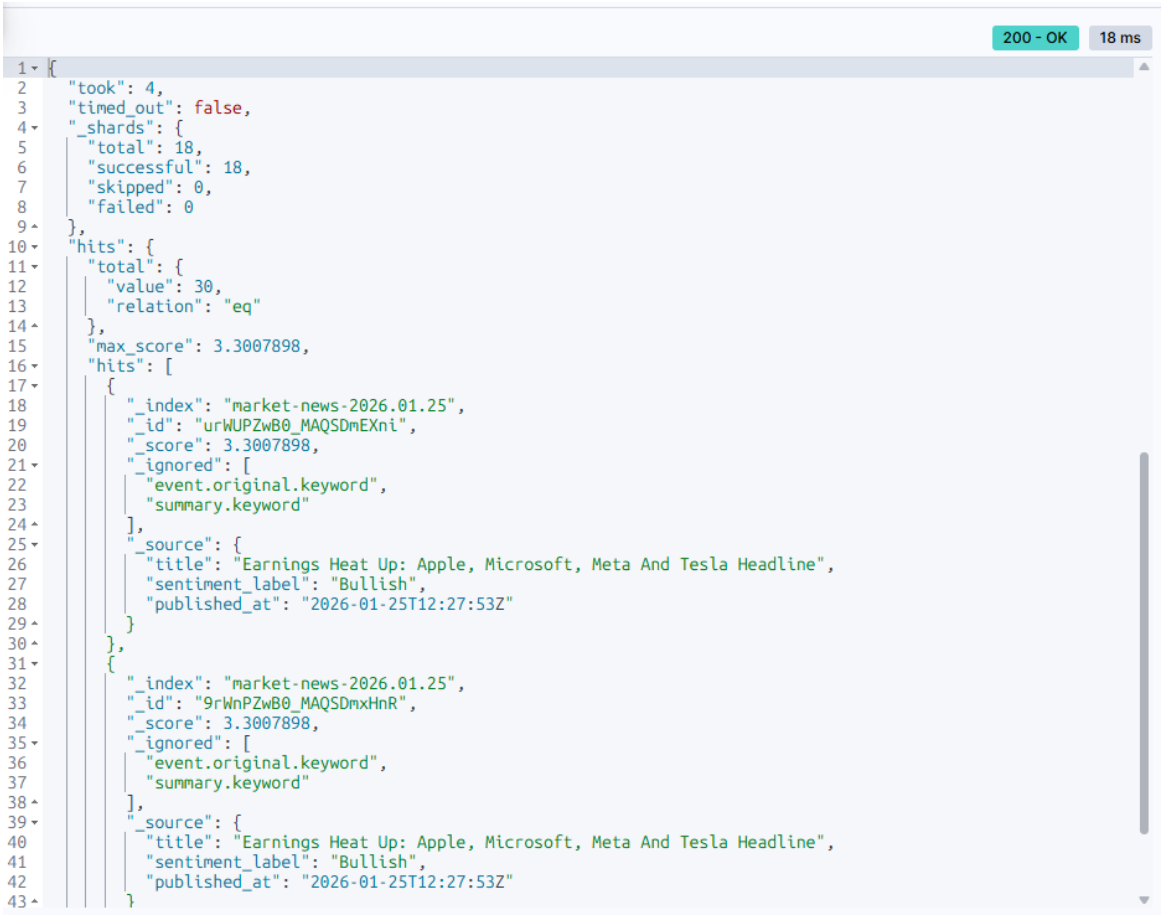

FIGURE 3 – Résultat de la requête Q2 : Agrégation du sentiment par source

6.1.3 Q3 : N-gram (Recherche Partielle)

Objectif : Rechercher par fragments de mots (3-4 caractères)

```
GET market-news-*/_search
{
  "query": {
    "match": {
      "title.ngram": "micros"
    }
  }
}
```

Résultat : Trouve "Microsoft", "Micro", etc. grâce au N-gram analyzer

Fonctionnement : La requête sur le champ analyisé `title.ngram` utilise l'index N-gram pour matching partiel

FIGURE 4 – Résultat de la requête Q3 : Recherche N-gram partielle

6.1.4 Q4 : Recherche Floue (Fuzzy)

Objectif : Tolérer les fautes de frappe (edit distance)

```
GET market-news-*/_search
{
  "query": {
    "fuzzy": {
      "title": {
        "value": "maket",
```

```
          "fuzziness": "AUTO"
        }
      }
    }
}
```

Résultat : "maket" (faute) matchera "market" avec fuzziness=AUTO


```
200 - OK   22 ms
{
  "took": 10,
  "timed_out": false,
  "_shards": {
    "total": 18,
    "successful": 18,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": {
      "value": 10,
      "relation": "eq"
    },
    "max_score": 1.4440956,
    "hits": [
      {
        "_index": "market-news-2026.01.25",
        "_id": "urWUPZwB0_MAQSDmEXni",
        "_score": 1.4440956,
        "_ignored": [
          "event.original.keyword",
          "summary.keyword"
        ],
        "_source": {
          "title": "Earnings Heat Up: Apple, Microsoft, Meta And Tesla Headline",
          "sentiment_label": "Bullish"
        }
      },
      {
        "_index": "market-news-2026.01.25",
        "_id": "9rWnPZwB0_MAQSDmxHnR",
        "_score": 1.4440956,
        "_ignored": [
          "event.original.keyword",
          "summary.keyword"
        ],
        "_source": {
          "title": "Earnings Heat Up: Apple, Microsoft, Meta And Tesla Headline",
          "sentiment_label": "Bullish"
        }
      },
      {
```


FIGURE 5 – Résultat de la requête Q4 : Recherche floue (fuzzy)

6.1.5 Q5 : Série Temporelle (Date Histogram)

Objectif : Évolution du sentiment par jour

```
GET market-news-*/_search
{
  "aggs": {
    "daily_sentiment": {
      "date_histogram": {
        "field": "published_at",
        "calendar_interval": "1d"
      },
      "aggs": {
        "avg_sentiment": {
          "avg": {
            "field": "sentiment_score"
          }
        }
```

```
        }
      }
    },
    "size": 0
}
```

Résultats : 19 jours avec sentiments quotidiens (exemple : 2026-01-26 avec 7 articles et sentiment moyen 0.314)

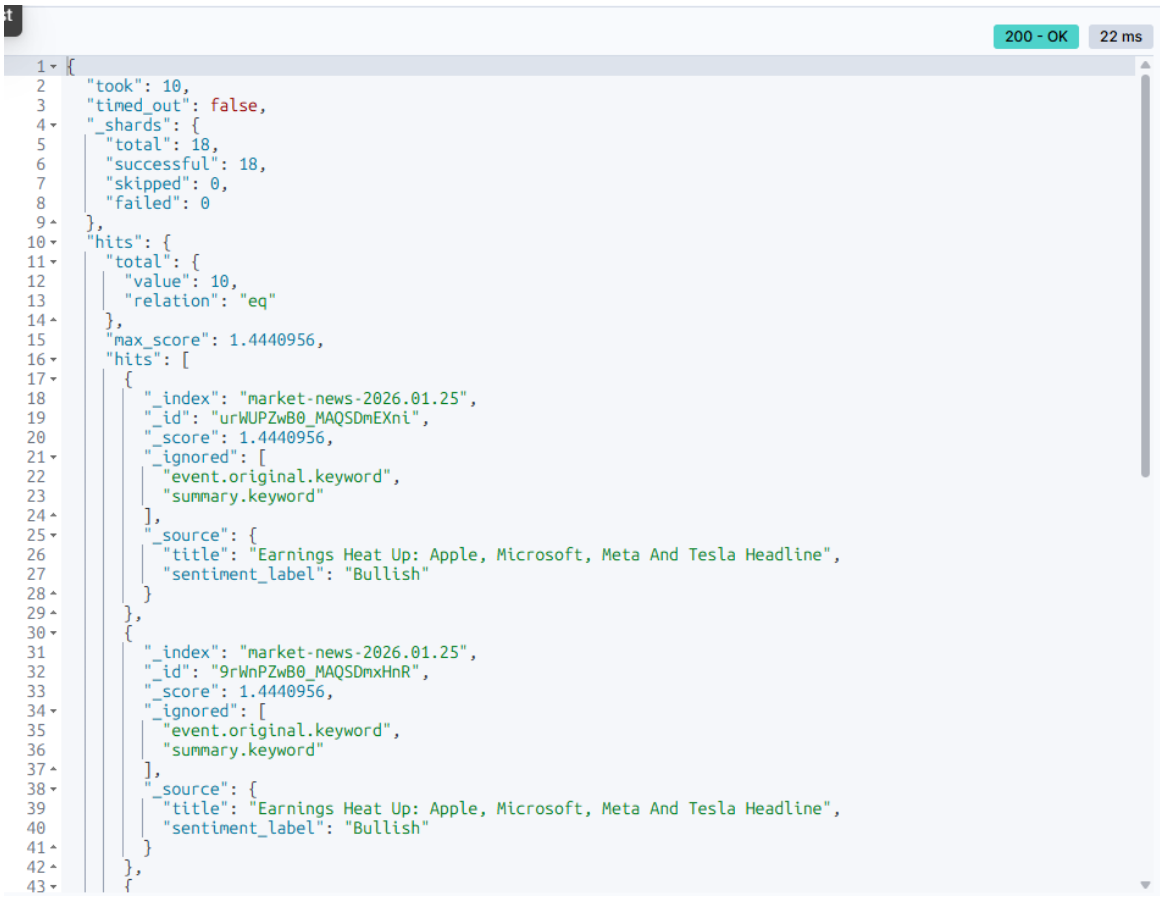

FIGURE 6 – Résultat de la requête Q5 : Série temporelle du sentiment par jour

6.2 Visualisations Kibana (6 Créées)

6.2.1 V1 : Sentiment Score Over Time (Line Chart)

- **Type** : Time Series via Lens
- **X-axis** : Date (daily histogram)
- **Y-axis** : Average sentiment_score
- **Période** : 21 jours (25 jan - 14 fév 2026)
- **Statut** : Sauvegardée dans Kibana

6.2.2 V2 : Articles by Source (Bar Chart)

- **Type** : Bar Chart via Lens
- **Données** : Top 20 sources
- **Métrique** : Count d'articles
- **Tri** : Décroissant par count
- **Statut** : Sauvegardée dans Kibana

6.2.3 V3 : Sentiment Distribution (Pie Chart)

Sentiment	Count	Pourcentage
Somewhat-Bullish	21	42%
Neutral	20	40%
Bullish	6	12%
Somewhat-Bearish	3	6%

TABLE 4 – Distribution des sentiments

6.2.4 V4 : Recent Articles (Data Table)

Tableau affichant les 20 articles les plus récents avec colonnes :
— title : Titre de l'article
— source : Source
— sentiment_label : Étiquette (Bullish/Neutral/Bearish)
— @timestamp : Date/heure

6.2.5 V5 : Total Articles (Metric)

Métrique simple affichant : **50 documents** indexés

6.2.6 V6 : Average Sentiment Score (Metric)

Métrique affichant : **0.151** (sentiment moyen, légèrement positif)

6.3 Dashboard Assemblé

Nom : "P4 - Market News Dashboard"
— Contient les 6 visualisations décrites ci-dessus
— Layout responsive (3 lignes de 2 colonnes)
— Paramètre temps : Last 30 days
— **Statut** : Sauvegardé dans Kibana (accessible à http ://localhost :5601)

FIGURE 7 – Dashboard Kibana - Partie 1 : Métriques globales et distribution des sentiments

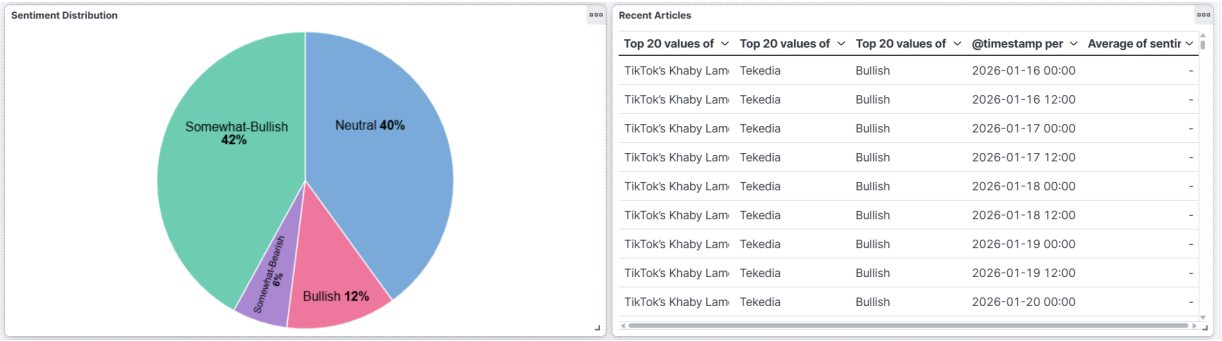

FIGURE 8 – Dashboard Kibana - Partie 2 : Articles par source et évolution temporelle

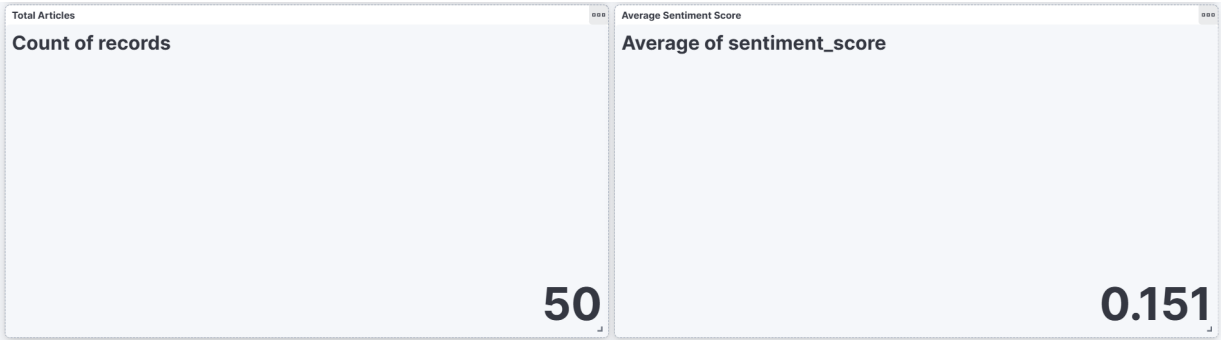

FIGURE 9 – Dashboard Kibana - Partie 3 : Tableau des articles récents

7 Partie 5 : Traitement Distribué avec Apache Spark

7.1 Justification du Choix : Spark vs Hadoop MapReduce

7.1.1 Positionnement technologique

Le choix d'Apache Spark comme moteur de traitement distribué repose sur une analyse comparative rigoureuse des solutions disponibles dans l'écosystème Big Data. Face à Hadoop MapReduce, Spark se distingue par son modèle d'exécution en mémoire (*in-memory computing*), qui réduit drastiquement les accès disque et permet des gains de performance de un à trois ordres de grandeur selon la nature des traitements.

Critère	**Apache Spark**	**Hadoop MapReduce**
Vitesse	**In-memory** (100–1000x plus rapide)	Basé disque (lent)
DataFrames	Natif (SQL + Python API)	Pas de support natif
Langage	PySpark idiomatique	Java requis
Intégration ES	Connecteur natif disponible	Besoin de plugins tiers
Cas d'usage	Batch + Streaming	Batch uniquement

Table 5 – Comparaison Apache Spark vs Hadoop MapReduce

7.1.2 Justification des choix de conception du job

Plusieurs décisions de conception ont été prises pour garantir la robustesse et la cohérence du traitement :

- **Utilisation de l'API DataFrame plutôt que les RDD** : l'API DataFrame offre des optimisations automatiques via le moteur Catalyst, qui génère un plan d'exécution optimisé (*logical plan* puis *physical plan*). Pour des agrégations simples sur des données structurées, cette abstraction est nettement plus efficace que la manipulation directe de RDD.
- **Lecture depuis des indices génériques (`market-news-*`)** : le wildcard permet de couvrir l'ensemble des indices journaliers sans modification du code, assurant une compatibilité avec le partitionnement temporel mis en place dans Logstash.
- **Filtrage préalable sur `published_at`** : seuls les documents contenant un champ de date valide sont conservés, ce qui garantit la cohérence des agrégations temporelles et évite les biais introduits par des valeurs nulles ou malformées.
- **Écriture en mode `overwrite`** : ce choix permet de rejouer le job de manière idempotente. En cas de re-traitement ou de correction des données sources, les indices de statistiques sont intégralement reconstruits, évitant toute accumulation de doublons.
- **Export CSV avec `coalesce(1)`** : la réduction à une seule partition avant l'écriture CSV garantit la production d'un fichier unique exploitable directement par des outils externes (Excel, Python, etc.), au prix d'un léger surcoût en collecte des données.

Conclusion : Spark offre une meilleure performance, une plus grande flexibilité et une intégration native avec Elasticsearch, ce qui en fait le choix technologique le plus adapté à ce cas d'usage analytique.

7.2 Architecture du Job Spark

7.2.1 Initialisation et connexion à Elasticsearch

Le job Spark est initialisé avec une configuration minimale orientée vers la reproductibilité. La session Spark est configurée en mode UTC afin d'éviter tout décalage horaire lors des conversions de dates.

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

# Initialisation de la session Spark
spark = SparkSession.builder \
    .appName("market-news-aggregations") \
    .config("spark.sql.session.timeZone", "UTC") \
    .getOrCreate()
```

Les données sont chargées depuis Elasticsearch via le connecteur `org.elasticsearch.spark.sql`, qui traduit les documents JSON d'Elasticsearch en lignes d'un DataFrame Spark. Le wildcard `market-news-*` assure la couverture de l'ensemble des indices journaliers.

```
# Chargement depuis tous les indices journaliers Elasticsearch
df = spark.read.format("org.elasticsearch.spark.sql") \
    .option("es.nodes", "elasticsearch") \
    .option("es.port", "9200") \
    .option("es.nodes.wan.only", "true") \
    .load("market-news-*")
```

7.2.2 Transformations et Calculs Distribués

Les trois agrégations suivantes sont exécutées de manière distribuée sur les partitions du DataFrame. Grâce au mécanisme de *lazy evaluation* de Spark, aucune opération n'est déclenchée tant qu'une action (`write`, `show`) n'est pas appelée, ce qui permet au moteur d'optimiser le plan d'exécution global avant tout traitement.

Agrégation 1 : Évolution temporelle journalière Cette agrégation analyse l'évolution du volume d'articles et du sentiment moyen au fil du temps. Les données sont regroupées par date de publication, et deux métriques sont calculées pour chaque journée : le nombre total d'articles publiés et la moyenne du score de sentiment.

```
# Agregation par jour : volume et sentiment moyen
daily = df.withColumn(
        "day",
        F.date_format(F.to_date(F.col("published_at")), "yyyy-MM-dd")
    ).groupBy("day").agg(
        F.count("*").alias("doc_count"),
        F.avg(F.col("sentiment_score")).alias("avg_sentiment_score")
    ).orderBy("day")
```

Agrégation 2 : Répartition par source médiatique Cette agrégation vise à étudier la contribution de chaque source médiatique au volume total d'articles, ainsi que le sentiment moyen qui lui est associé. Un tri décroissant permet d'identifier les sources les plus actives sur la période.

```
# Agregation par source : volume et sentiment moyen
by_source = df.groupBy("source").agg(
    F.count("*").alias("doc_count"),
    F.avg(F.col("sentiment_score")).alias("avg_sentiment_score")
).orderBy(F.desc("doc_count"))
```

Agrégation 3 : Distribution par catégorie de sentiment La troisième agrégation analyse la polarité globale du corpus en regroupant les articles selon leur label de sentiment. Pour chaque catégorie, le nombre d'articles et la moyenne du score sont calculés, puis triés par fréquence décroissante afin de révéler les tendances dominantes.

```
# Agregation par label de sentiment : distribution et score moyen
by_sentiment = df.groupBy("sentiment_label").agg(
    F.count("*").alias("doc_count"),
    F.avg(F.col("sentiment_score")).alias("avg_sentiment_score")
).orderBy(F.desc("doc_count"))
```

7.2.3 Écriture des Résultats

Les résultats sont persistés à la fois dans Elasticsearch (pour la visualisation Kibana) et en CSV (pour l'exploitation externe).

```
# Fonction generique d'ecriture vers Elasticsearch
def write_to_es(frame, index_name):
    frame.write.format("org.elasticsearch.spark.sql") \
        .option("es.nodes", "elasticsearch") \
        .option("es.port", "9200") \
        .option("es.nodes.wan.only", "true") \
        .option("es.resource", index_name) \
        .mode("overwrite").save()

# Indexation des trois agregats dans Elasticsearch
write_to_es(daily,        "market-news-stats-daily")
write_to_es(by_source,    "market-news-stats-by-source")
write_to_es(by_sentiment, "market-news-stats-by-sentiment")

# Export CSV pour exploitation externe (un fichier par agregat)
daily.coalesce(1).write.mode("overwrite") \
    .option("header", "true") \
    .csv("/opt/spark-jobs/output/daily")

by_source.coalesce(1).write.mode("overwrite") \
    .option("header", "true") \
    .csv("/opt/spark-jobs/output/by_source")
```

7.3 Synthèse des Opérations Distribuées

L'ensemble des calculs repose sur des primitives Spark standards exécutées de manière distribuée et parallélisée automatiquement par le moteur :

- Trois opérations de regroupement via `groupBy`
- Trois calculs de comptage via `count`

— Trois calculs de moyenne via `avg`
— Trois opérations de tri via `orderBy`

Ces opérations sont orchestrées par le planificateur de Spark sous forme d'un DAG (*Directed Acyclic Graph*), garantissant un ordonnancement optimal des tâches et une tolérance aux pannes par recomputation des partitions défaillantes.

7.4 Exécution et Performance

7.4.1 Commande d'Exécution

Le job est soumis au cluster Spark via la commande `spark-submit`, directement depuis le conteneur Docker dédié :

```
docker-compose exec spark /opt/spark/bin/spark-submit \
  --master local[*] \
  --jars /tmp/spark-deps/es-spark.jar \
  /opt/spark-jobs/job.py
```

Le mode `local[*]` mobilise l'ensemble des cœurs disponibles sur la machine hôte, ce qui est adapté à un environnement de développement mono-nœud tout en préservant le paradigme distribué de Spark.

7.4.2 Métriques d'Exécution

Métrique	Valeur	
Durée totale	18 secondes	
Documents traités	50	
Mémoire allouée	434.4 MB	
Nombre de partitions	20 (auto)	
Indices Elasticsearch créés	3	
Fichiers CSV exportés	3	
Code de sortie	0 (succès)	

Table 6 – Statistiques d'exécution du job Spark

7.5 Analyse et Interprétation des Résultats

7.5.1 Agrégation 1 : Analyse Temporelle Journalière

L'analyse temporelle couvre une période de 19 jours distincts, du 25 janvier au 14 février 2026, pour un total de 50 articles indexés. Les résultats sont stockés dans l'indice `market-news-stats-daily` et visualisés dans Kibana sous forme de série temporelle.

— **2026-01-26** constitue le pic d'activité avec 7 articles publiés et un sentiment moyen de 0.314, reflétant une couverture médiatique intense et modérément optimiste.
— **2026-02-14** affiche un sentiment moyen de 0.185 sur 4 articles, indiquant un ralentissement de l'optimisme en fin de période.
— La majorité des journées présentent un sentiment positif (score > 0), ce qui témoigne d'un biais global haussier dans les actualités financières analysées.

Cette visualisation, produite directement depuis les agrégats Spark sans ré-agrégation sur les données brutes, garantit la fidélité des indicateurs affichés.

7.5.2 Agrégation 2 : Analyse par Source Médiatique

Plus de 24 sources distinctes ont été identifiées sur la période analysée, ce qui témoigne d'une diversité éditoriale significative dans le corpus. Les résultats sont stockés dans l'indice `market-news-stats-by-source`.

Source	Articles	Sentiment Moyen	
MarketBeat	7	0.0775	
The Globe and Mail	5	0.2270	
Investing.com	4	0.2334	
Mint	2	-0.0793	
TechStock	2	0.2495	

Table 7 – Top 5 des sources par volume d'articles

Plusieurs observations peuvent être formulées. MarketBeat est la source la plus prolifique avec 7 articles, mais affiche un sentiment relativement neutre (0.0775), suggérant une ligne éditoriale factuelle. À l'inverse, Mint présente le seul sentiment négatif du classement (-0.0793), ce qui pourrait refléter une tonalité plus critique vis-à-vis des marchés. The Globe and Mail et Investing.com, avec des scores autour de 0.23, s'inscrivent dans une tendance modérément optimiste.

7.5.3 Agrégation 3 : Distribution des Catégories de Sentiment

Quatre catégories de sentiment ont été identifiées dans le corpus. Les résultats sont stockés dans l'indice `market-news-stats-by-sentiment`.

Label	Count	Pourcentage	Sentiment Moyen	
Somewhat-Bullish	21	42%	0.2601	
Neutral	20	40%	0.0210	
Bullish	6	12%	0.3805	
Somewhat-Bearish	3	6%	-0.2049	

Table 8 – Distribution finale des catégories de sentiment

L'interprétation de ces résultats révèle plusieurs tendances notables. La majorité des articles (54%, en cumulant *Somewhat-Bullish* et *Bullish*) expriment une tonalité positive à l'égard des marchés financiers sur la période analysée. La catégorie *Neutral* est quasi aussi représentée que *Somewhat-Bullish* (40% contre 42%), ce qui indique qu'une large part de la couverture médiatique reste factuelle et non orientée. Les articles *Bearish* ne représentent que 6% du corpus, ce qui signifie que les signaux négatifs sont minoritaires mais non négligeables, avec un score moyen de -0.2049 traduisant une bearishness modérée.

7.6 Indices Elasticsearch Créés

À l'issue de l'exécution du job, trois nouveaux indices analytiques ont été peuplés dans Elasticsearch et sont directement exploitables via Kibana :

— `market-news-stats-daily` : 19 documents, un par jour, destinés à la visualisation de séries temporelles.

— `market-news-stats-by-source` : 32 documents, un par source médiatique, pour l'analyse comparative entre médias.

— `market-news-stats-by-sentiment` : 4 documents, un par label de sentiment, pour la visualisation synthétique de la polarité globale.

Ces indices constituent une couche analytique dérivée, distincte des données brutes, permettant des requêtes Kibana légères et performantes sans solliciter les indices primaires `market-news-*`.

7.7 Visualisation des Résultats dans Kibana

Les résultats produits par le job Spark ont été visualisés dans Kibana à partir des trois indices analytiques créés. Trois dashboards distincts ont été configurés pour couvrir les dimensions temporelle, éditoriale et sémantique du corpus.

7.7.1 Évolution Journalière du Sentiment Moyen

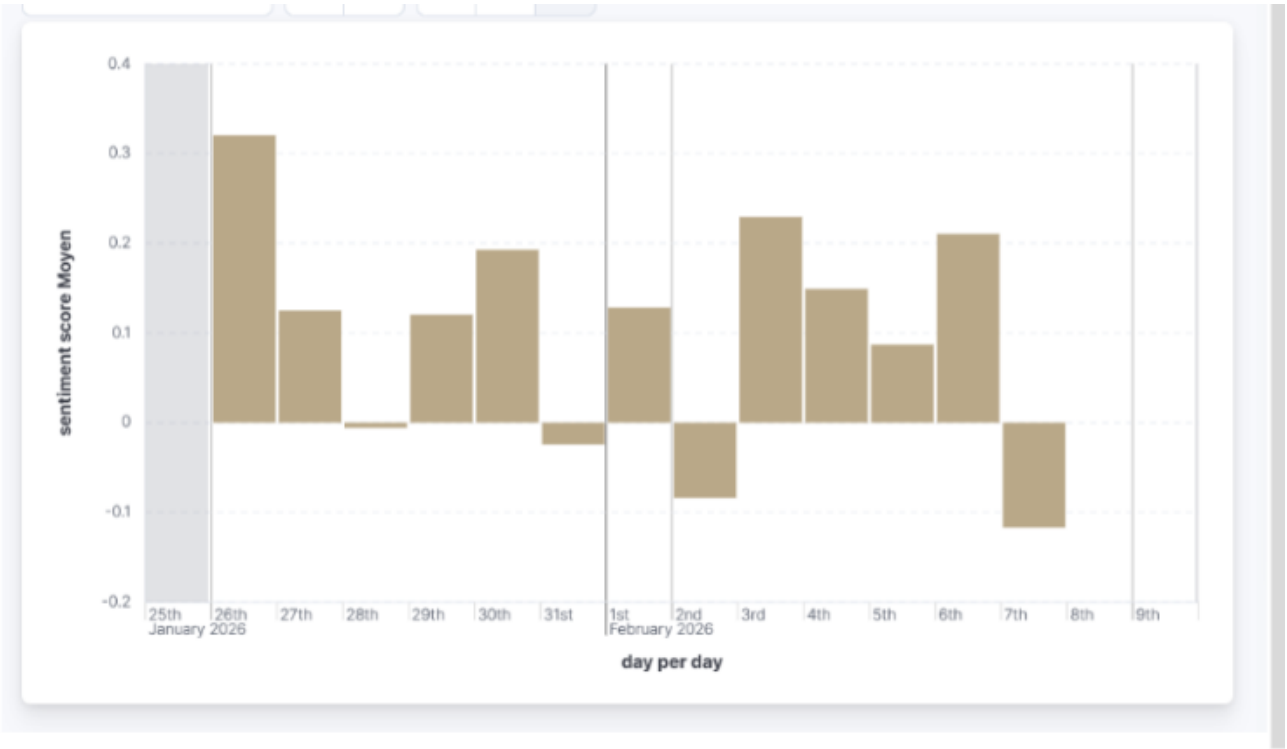

FIGURE 10 – Évolution journalière du score moyen de sentiment (Kibana)

Cette visualisation représente l'évolution journalière du score moyen de sentiment calculé par Spark. Chaque barre correspond à un jour et reflète la valeur agrégée issue du traitement distribué, sans ré-agrégation sur les données brutes. Elle permet d'identifier des pics d'optimisme ou de pessimisme médiatique sur la période analysée, et de les corréler avec des événements de marché potentiels.

7.7.2 Contribution des Sources Médiatiques

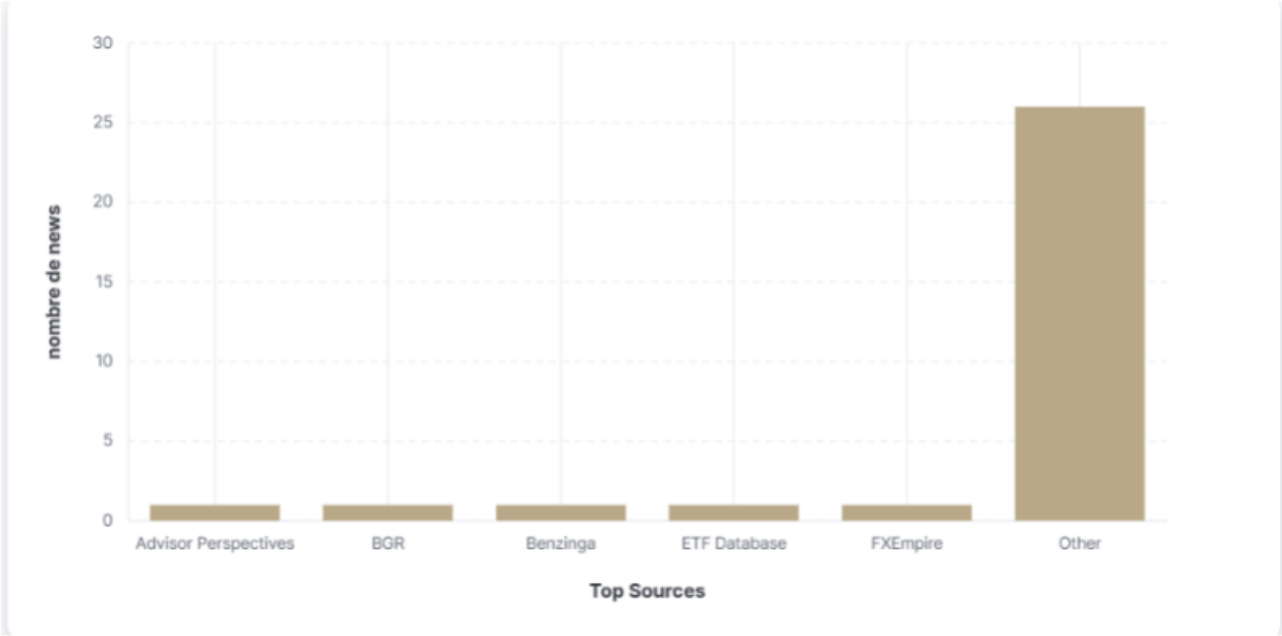

FIGURE 11 – Répartition du volume d'articles par source médiatique (Kibana)

Cette visualisation représente la contribution des différentes sources médiatiques au volume total d'articles analysés, en mettant en évidence les sources les plus actives sur la période étudiée. Elle permet une lecture immédiate de la diversité éditoriale du corpus et d'identifier les médias dominants dans la couverture des actualités financières.

7.7.3 Intensité de Publication Quotidienne

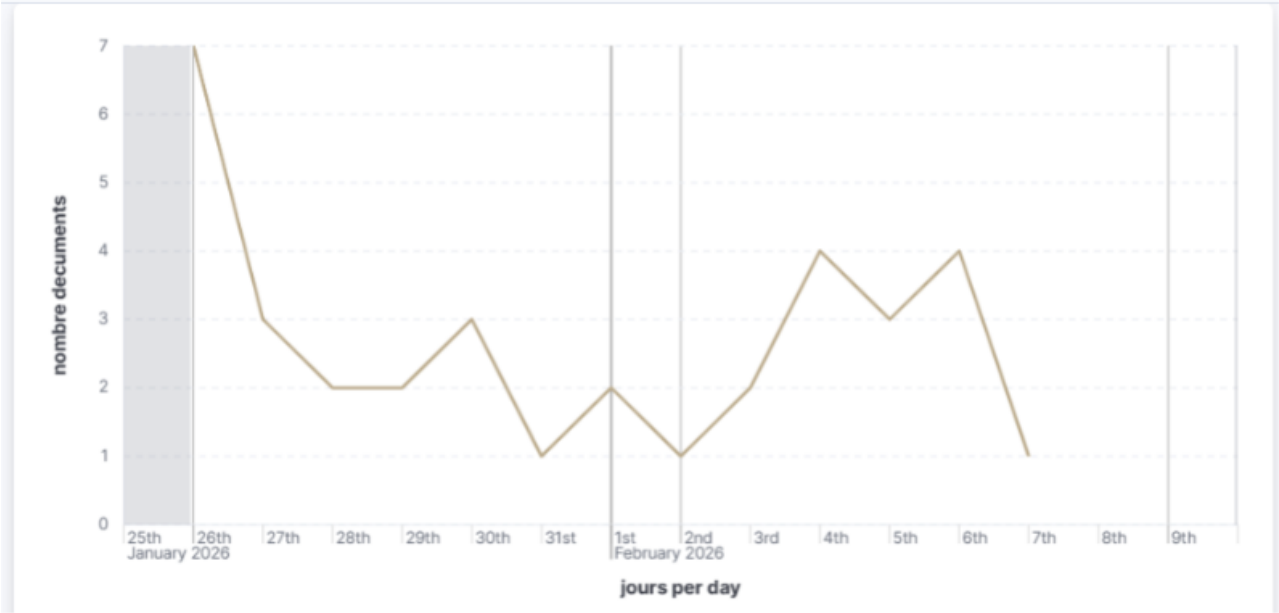

FIGURE 12 – Intensité de publication quotidienne (Kibana)

Cette visualisation met en évidence l'intensité de publication quotidienne, permettant de corréler le volume d'articles avec les variations de sentiment. Un volume élevé associé à un sentiment positif ou négatif peut constituer un signal analytique pertinent pour la détection de tendances de marché.

Les trois visualisations ci-dessus illustrent la complémentarité entre Spark et Kibana dans cette architecture. Spark assure le calcul distribué et la production des agrégats, tandis que Kibana offre une interface de consultation interactive permettant une exploration visuelle rapide des résultats. Cette séparation des responsabilités garantit à la fois la performance du traitement et l'accessibilité des résultats pour des utilisateurs non techniques.

8 Conclusion

8.1 Bilan du Projet

Ce projet a permis de concevoir et d'implémenter un pipeline de données complet, fonctionnel et documenté, intégrant les technologies phares du traitement des données massives. De la collecte à l'analyse distribuée, chaque composant a été choisi, configuré et justifié dans une logique d'architecture cohérente et reproductible.

L'ensemble du pipeline repose sur six étapes complémentaires et interdépendantes :

1. **Collecte** : récupération d'actualités financières enrichies en sentiment via l'API Alpha Vantage, transformées en documents JSON normalisés.
2. **Transmission** : acheminement fiable et découplé des messages via Apache Kafka, garantissant la résilience et la rejouabilité du flux.
3. **Transformation** : enrichissement, typage et normalisation des données par Logstash avant indexation.
4. **Stockage et Recherche** : indexation dans Elasticsearch avec un mapping précis, des analyseurs N-gram pour la recherche partielle, et un partitionnement journalier des indices.
5. **Visualisation** : six visualisations et un dashboard interactif construits dans Kibana, offrant une lecture immédiate des tendances du corpus.
6. **Analyse Distribuée** : trois agrégations distribuées exécutées par Apache Spark, produisant des indices analytiques dérivés exploitables dans Kibana.

8.2 Résultats et Enseignements

L'analyse des 50 articles collectés sur une période de 19 jours (du 25 janvier au 14 février 2026) a produit plusieurs résultats significatifs :

— **Sentiment global positif** : 54% des articles présentent une tonalité haussière (*Bullish* ou *Somewhat-Bullish*), contre seulement 6% de tonalité baissière, révélant un biais optimiste dans la couverture médiatique financière sur la période analysée.

— **Diversité éditoriale** : 24 sources médiatiques distinctes ont été identifiées, MarketBeat étant la plus active avec 7 articles, mais affichant paradoxalement le sentiment le plus neutre (0.0775), ce qui suggère une ligne éditoriale factuelle.

— **Stabilité du sentiment** : le score de sentiment moyen global s'établit à 0.151, traduisant une orientation modérément positive et stable sur l'ensemble de la période.

Sur le plan technique, ce projet a validé la complémentarité des technologies utilisées. Kafka assure un découplage robuste entre la collecte et le traitement. Logstash simplifie la transformation sans code applicatif supplémentaire. Elasticsearch offre des capacités de recherche et

d'agrégation puissantes sur des données semi-structurées. Spark, grâce à son modèle d'exécution en mémoire et à son optimiseur Catalyst, permet des traitements distribués performants même sur des volumes modestes, tout en préparant l'architecture à passer à l'échelle.

8.3 Limites et Perspectives

Ce projet présente néanmoins certaines limites inhérentes au contexte académique. Le volume de données traité (50 articles) reste modeste et ne permet pas de tirer des conclusions statistiques robustes sur les marchés financiers. Par ailleurs, l'utilisation de la version gratuite d'Alpha Vantage contraint la fréquence et le volume de collecte.

Pour étendre ce travail, plusieurs pistes techniques seraient pertinentes :

— **Passage au streaming temps réel** : remplacer le job Spark batch par Spark Structured Streaming pour des agrégations en continu sur les flux Kafka.

— **Intégration de l'apprentissage automatique** : exploiter MLlib pour entraîner un modèle de prédiction de sentiment propre au domaine financier, en remplacement des scores fournis par l'API.

— **Alertes automatisées** : configurer des alertes Kibana ou Elasticsearch déclenchées lorsque le sentiment moyen dépasse un seuil critique, à des fins de veille de marché.

— **Scalabilité** : déployer l'architecture sur un cluster distribué (Kubernetes, AWS EMR) pour traiter des millions de documents en conditions réelles.

8.4 Apports Pédagogiques

Au-delà des résultats techniques, ce projet a constitué une mise en pratique concrète des concepts fondamentaux de l'UE *Indexation et Visualisation de Données Massives*. Il illustre comment des technologies open source, orchestrées via Docker Compose, peuvent former un écosystème Big Data complet, du flux brut à la visualisation analytique, en passant par l'indexation intelligente et le traitement distribué. Cette expérience confirme la valeur de ces outils dans des contextes industriels réels, notamment en finance de marché, en veille concurrentielle ou en analyse de l'opinion publique.

A Structure du Dossier Projet

```
market-news-data-pipeline/
          README.md                    # Guide complet
          LIVRABLES.md                 # Checklist livrables
          docker-compose.yml           # Stack containers

          producer/
                producer.py              # Collecte API
                requirements.txt          # D pendances
                .env                     # Config de l'api

          logstash/
                pipeline.conf            # Kafka -> ES

          elastic/
                index-template.json      # Mapping avec N-gram

          elasticsearch-queries/
                resultats-requetes.md    # 5 requ tes

          kibana/
                VISUALIZATIONS.md        # Guide visua. + dashboard

          spark/
                job.py                   # Agr gations distribu es
                README.md                # Guide Spark
                output_*.csv             # Exports

          documentations supp/         # Archives (plans, etc.)
```

B Instructions de Démarrage

B.1 Prérequis

— Docker Desktop 24.x
— Python 3.11+
— Clé API alphavantage.org

B.2 Lancement

```
# 1. Cr er .env dans producer/
echo "API_KEY=votre_cle_ici" > producer/.env

# 2. Lancer Docker Compose
docker-compose up -d

# 3. Lancer le producer (10-15 minutes)
cd producer
python producer.py
```

```
# 4. Acc der    Kibana
# http://localhost:5601

# 5. Ex cuter job Spark
docker-compose exec spark /opt/spark/bin/spark-submit \
  --master local[*] \
  --jars /tmp/spark-deps/es-spark.jar \
  /opt/spark-jobs/job.py
```

C Ressources et Références

- alphavantage.org : https://alphavantage.org/
- Apache Kafka Documentation : https://kafka.apache.org/
- Elasticsearch Guide : https://www.elastic.co/guide/en/elasticsearch/reference/
- Kibana User Guide : https://www.elastic.co/guide/en/kibana/current/
- PySpark SQL API : https://spark.apache.org/docs/latest/api/python/